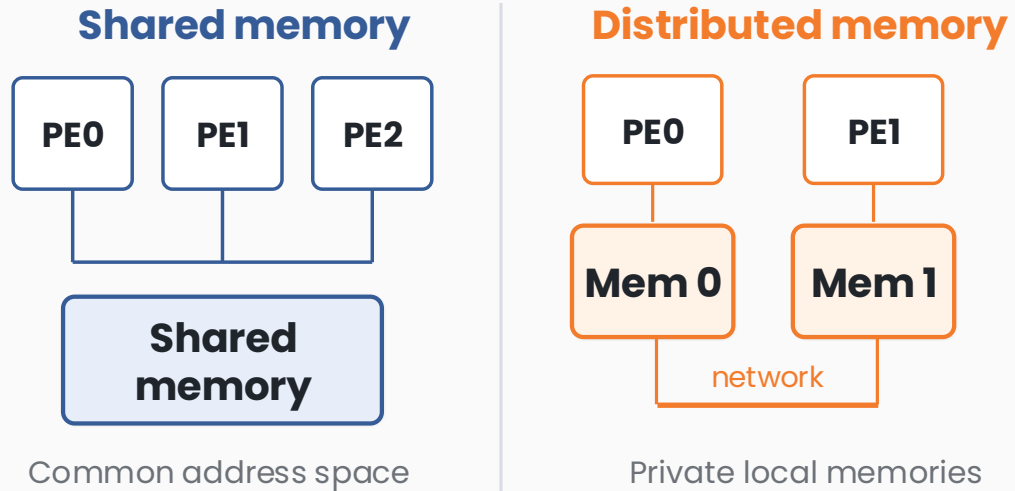
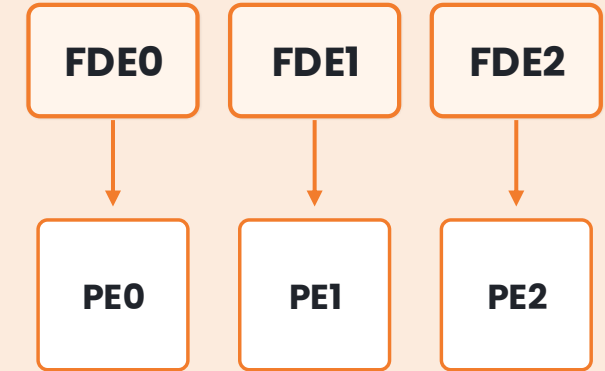


From one datapath to parallel architectures

Memory organization



Independent control + datapath



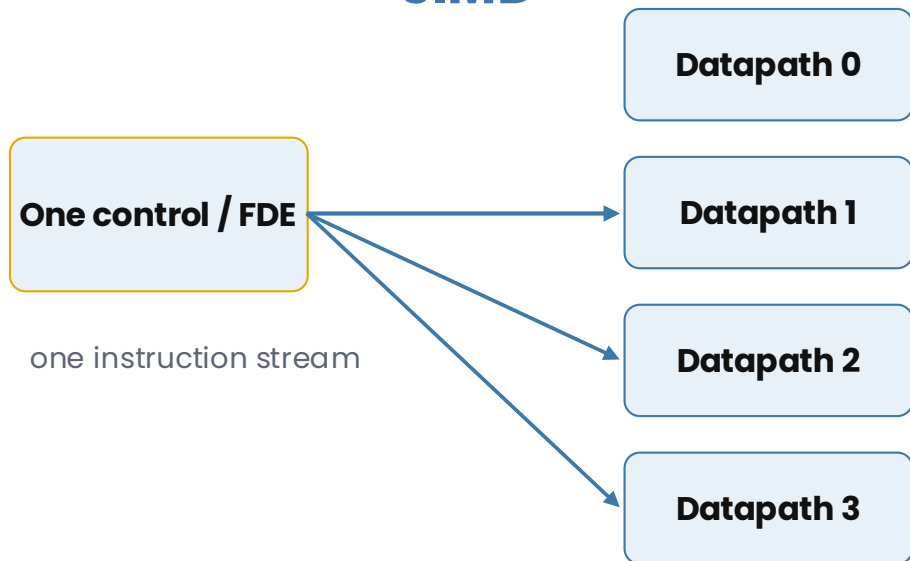
Each processing unit can progress through its own instruction stream.

MIMD

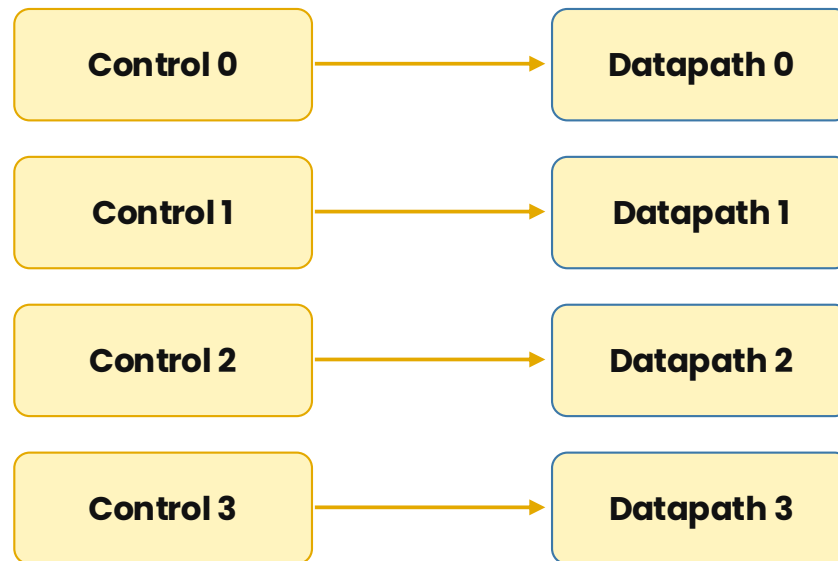
Independent dimensions: a MIMD machine can be shared-memory or distributed-memory; the memory model does not define the Flynn class.

From SIMD to MIMD

SIMD



MIMD



MIMD replicates both control and execution resources: each processing element can follow its own instruction stream.

What is MIMD?

- A MIMD system contains multiple independent processing elements.
- Each processing element has its own control unit, instruction stream, datapath, and execution state.
- Different processing elements may execute different instructions at the same time.
- They may also operate on different data.

Example: independent work

Core 0 $A[i] = B[i] + C[i]$

Core 1 `sort(x)`

Core 2 `filter(y)`

Core 3 `compress(z)`

“Multiple instructions” means independent instruction streams — not that every core must always execute a different instruction.

Example



Coordination only when needed

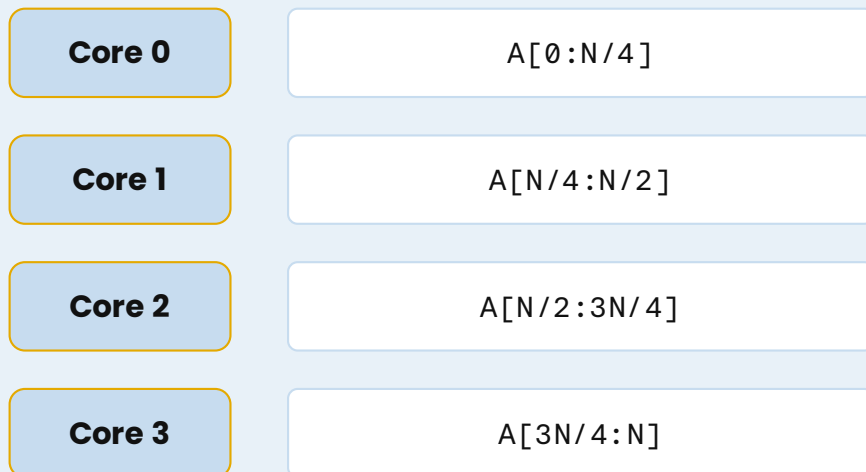
- barriers
- locks / atomics
- messages / events

MIMD execution is generally asynchronous: synchronization is an algorithmic requirement, not a defining lockstep behavior.

Data Parallelism vs Task Parallelism

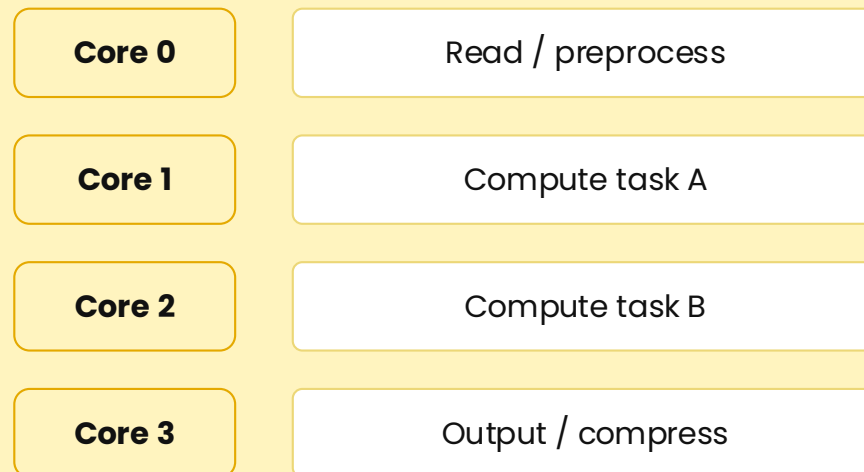
Data parallelism

Same program, different portions of the data



Task parallelism

Different computations can progress concurrently

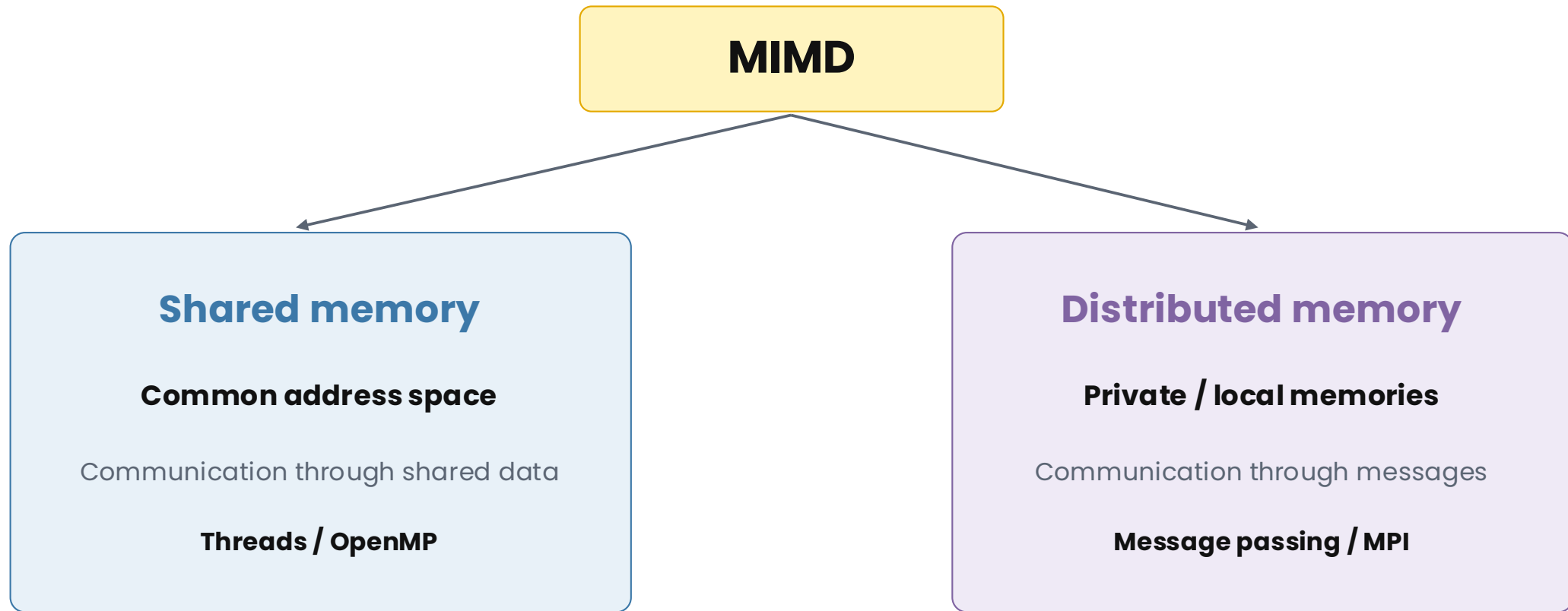


Unlike SIMD, MIMD does not require all processing elements to perform the same operation simultaneously.

MIMD Systems: How Is Memory Organized?

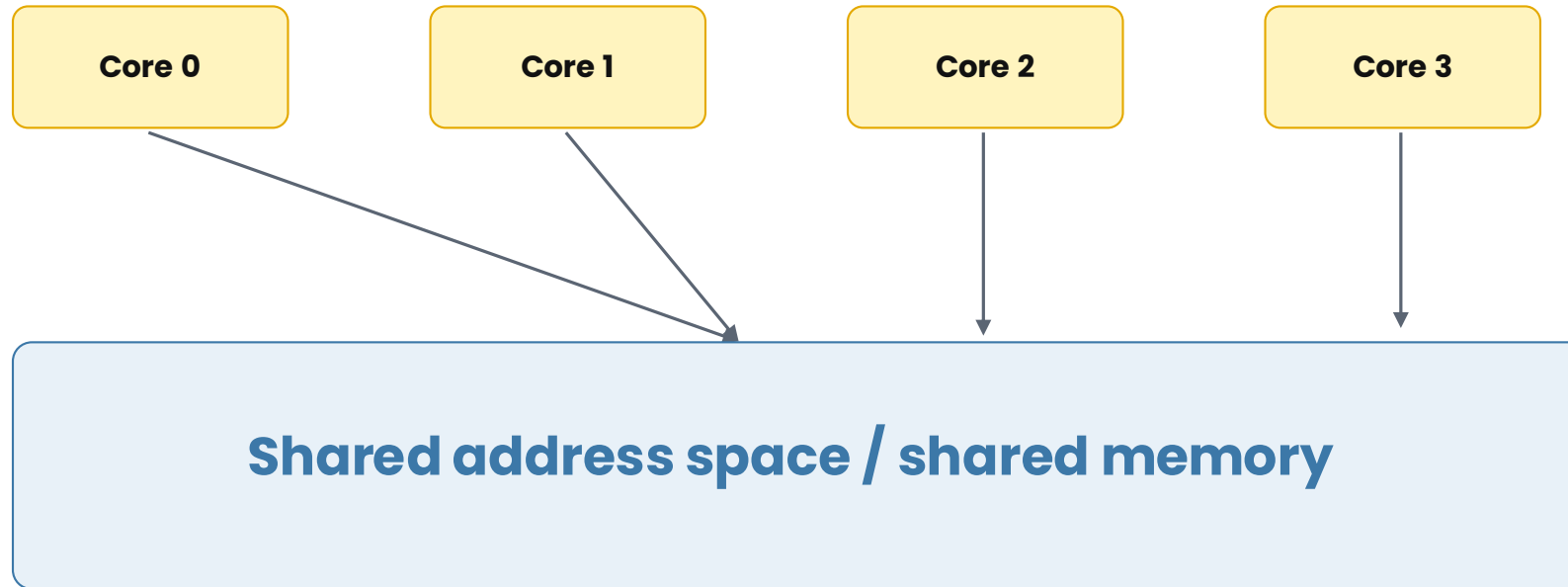


UNITRENTO



Shared vs distributed memory is not another Flynn category — it is an independent organization of the memory system.

Shared-memory MIMD



Key properties

- Each core has its own instruction stream.
- Cores communicate by reading/writing shared data.
- Synchronization protects shared state.

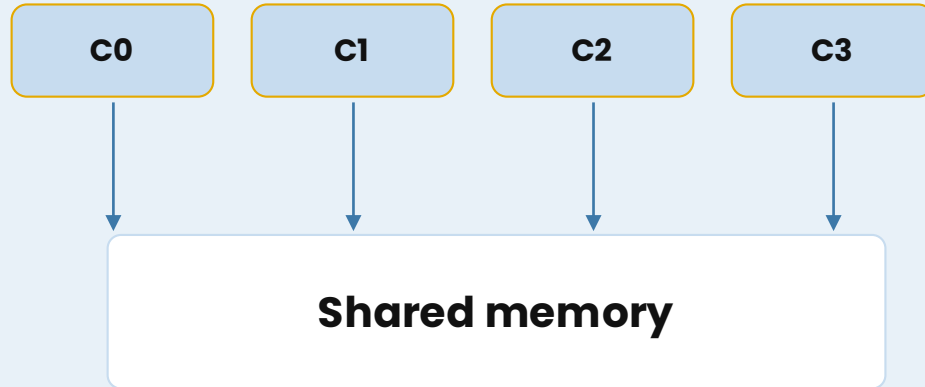
Independent cores communicate through a common address space.

Execution model: MIMD • Memory organization: shared memory

Shared Memory: UMA and NUMA

UMA

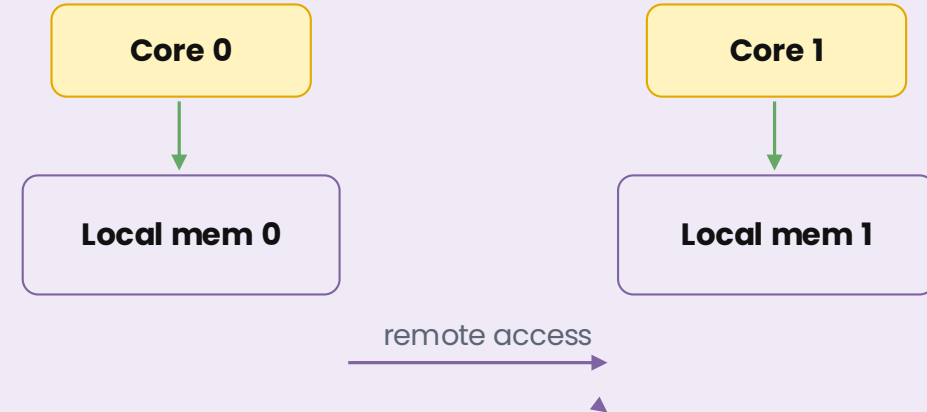
Uniform Memory Access



Access cost is approximately independent of which core accesses a location.

NUMA

Non-Uniform Memory Access



One address space, but local and remote memory have different access costs.

- **Both provide a shared address space, but memory access costs differ.**
- **NUMA locality, placement, and bandwidth effects will be discussed in the lab and memory subsystem.**

Shared Memory Requires Coordination

Both cores execute:

```
counter = counter + 1;
```

Core 0

Core 1

Is the result always correct?

Synchronization mechanisms

Locks

Mutual exclusion

Atomics

Indivisible updates

Barriers

Phase coordination

- A common address space makes communication easy — but shared updates can race.
- Cache coherence (mechanism to keep consistent the memory) and false sharing are important consequences of shared memory

Shared Memory: Advantages and Limitations



UNITRENTO

Advantages

- Common address space
- Natural data sharing
- Communication via ordinary loads/stores
- Convenient programming model

Challenges

- Synchronization is required
- Contention can limit scalability
- Access costs may be non-uniform
- Coherence becomes harder as systems grow

Programming simplicity does not imply uniform performance.

Multiple Instruction Multiple Data



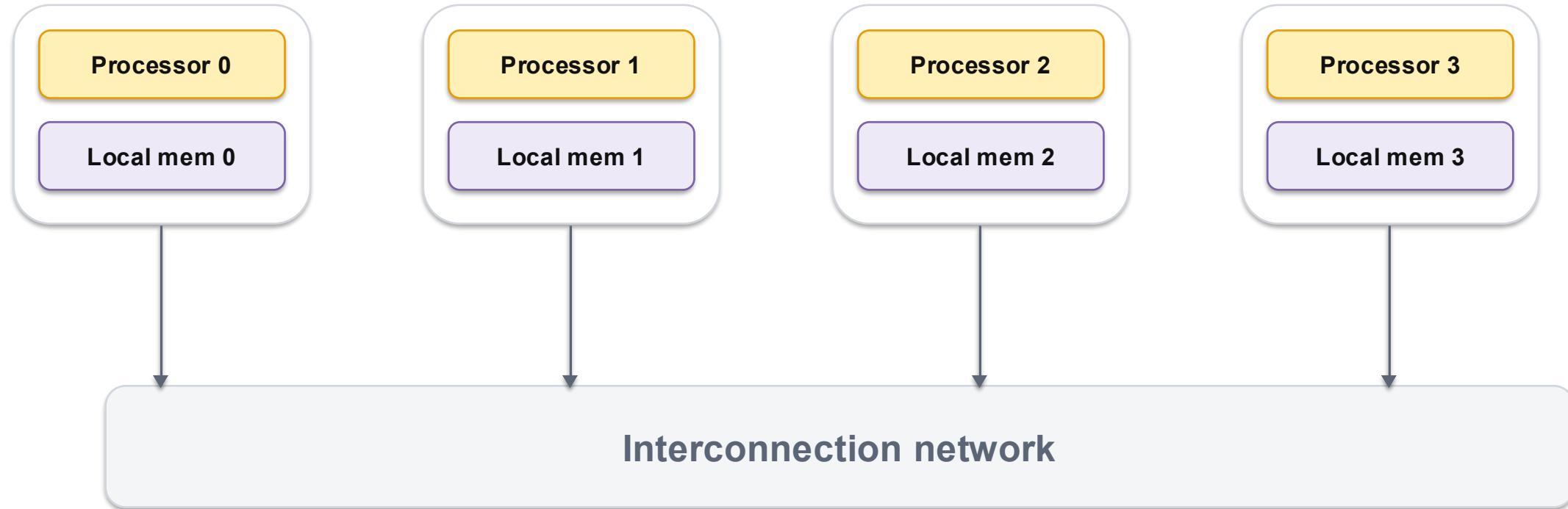
UNITRENTO

- Multiple fetch units: more instructions per data operation
- MIMD includes both **shared-memory (with threads)** and **distributed-memory (with MPI)** systems

MIMD architectures can exploit significant **data-level and task-level parallelism**

MIMD allows programmers to design task-oriented applications and different programming models (e.g., graph-based)

Distributed-memory MIMD



Non-local data must be communicated — there is no single hardware shared address space across nodes.

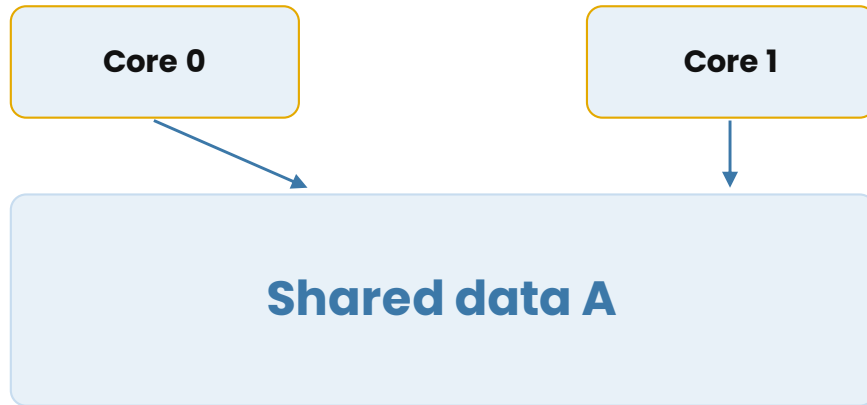
Each processing element/node has local memory and communicates over a network.

Distributed Memory: Explicit Communication

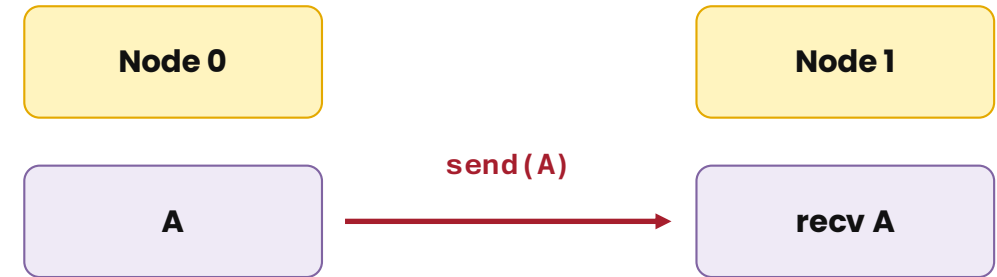


UNITRENTO

Shared memory



Distributed memory



**MPI is the dominant standard API
for message-passing HPC programs.**

Distributed memory makes communication explicit in the program.

Shared- vs distributed-memory

	Shared memory	Distributed memory
Address space	Shared	Private/local per node
Communication	Loads / stores	Messages over the network
Typical model	Threads / OpenMP	MPI
Synchronization	Locks, atomics, barriers	Messages, collectives and barriers
Scaling intuition	Limited by shared resources	Can scale across many nodes

Large HPC systems often combine both: shared memory inside a node and distributed memory across nodes.